Olympus Property Analytics — ETL Pipeline Technical Document

Version: 1.0 **Date:** March 2026 **System:** Olympus-property-analytics-master **Orchestration:** Prefect 2.x **Target Databases:** Microsoft SQL Server (`Olympus_Property_Staging`) | PostgreSQL (`Olympus_Property`)

Table of Contents

1. Overview
2. System Architecture
3. Technology Stack
4. Data Sources
5. ETL Pipeline Phases
   - Phase 1: Extract & Stage (Concurrent)
   - Phase 2: Dimension Population (Sequential)
   - Phase 3: Fact Table Population (Concurrent)
6. Load Task Inventory
7. Stored Procedures Reference
8. Data Transformation Classes
9. Database Connection Layer
10. Scheduling & Deployment
11. Directory Structure
12. Environment Configuration
13. Error Handling & Logging
14. Data Flow Diagram

---

1. Overview

The Olympus Property Analytics ETL pipeline is a daily-scheduled data engineering system that consolidates property management data from **30+ external source files** across **10+ operational systems** into a unified dimensional data warehouse. The pipeline powers Power BI dashboards and revenue management analytics for Olympus Property.

Key Metrics:

- **30+ concurrent load tasks** (staging phase)
- **5 sequential dimension procedures** (dimension phase)
- **40+ concurrent fact procedures** (fact phase)
- **~30–60 minutes** total end-to-end execution per run
- **Daily schedule** (runs once per day via Prefect)
- **Dual database support:** MS SQL Server and PostgreSQL run in parallel

---

2. System Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                          DATA SOURCES                                 │
│                                                                       │
│  Ellis PMS │ Knock │ Finance │ Banner │ SOCI │ Yelp │ Google │ ...   │
│  (CSV/Excel/XML files on network drives & local paths)               │
```

```
└──────────────────────────┬──────────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────────┐
│              PREFECT ORCHESTRATION LAYER                         │
│                                                                  │
│   workflow_tree_postgres.py  ──────  workflow_tree.py            │
│   (PostgreSQL Flow)                  (MS SQL Flow)               │
│                                                                  │
│   Work Pool: olympus_workpool  │  Queue: olympus_workqueue       │
│   Schedule: Daily (86400s)     │  Infrastructure: Process        │
└──────────────┬──────────────────────────────────┬──────────────┘
               │                                  │
               ▼                                  ▼
┌──────────────────────┐            ┌──────────────────────────┐
│   PHASE 1: STAGING   │            │   PHASE 1: STAGING       │
│   (ConcurrentTaskRunner│          │   (ConcurrentTaskRunner) │
│   30+ tasks in parallel│          │   30+ tasks in parallel) │
└──────────────┬──────────────────────┘            └──────────────┬──────────────────────┘
               │                                  │
               ▼                                  ▼
┌──────────────────────┐            ┌──────────────────────────┐
│   PHASE 2: DIMENSIONS  │          │   PHASE 2: DIMENSIONS    │
│   (SequentialTaskRunner│          │   (SequentialTaskRunner) │
│   5 stored procedures) │          │   5 stored procedures)   │
└──────────────┬──────────────────────┘            └──────────────┬──────────────────────┘
               │                                  │
               ▼                                  ▼
┌──────────────────────┐            ┌──────────────────────────┐
│   PHASE 3: FACTS     │            │   PHASE 3: FACTS         │
│   (ConcurrentTaskRunner│          │   (ConcurrentTaskRunner) │
│   40+ stored procedures│          │   40+ stored procedures) │
└──────────────┬──────────────────────┘            └──────────────┬──────────────────────┘
               │                                  │
               ▼                                  ▼
┌──────────────────────┐            ┌──────────────────────────┐
│  MS SQL SERVER       │            │  POSTGRESQL              │
│  Olympus_Property_   │            │  Olympus_Property        │
│  Staging DB          │            │  (schema: staging)       │
│  ├── Staging Tables  │            │  ├── Staging Tables      │
│  ├── Dimension Tables │           │  ├── Dimension Tables    │
│  └── Fact Tables     │            │  └── Fact Tables         │
└──────────────┬──────────────────────┘            └──────────────┬──────────────────────┘
               │                                  │
               └──────────────────┬──────────────────────┘
                                  ▼
                     ┌──────────────────────┐
                     │  Power BI Dashboards │
                     │  Revenue Analytics   │
                     │  Operational Reports │
```

```
                    │  CSV Exports (push_*.py)│
                    └────────────────────────┘
```

---

3. Technology Stack

Component	Technology	Version
Orchestration	Prefect	2.10.20
Language	Python	3.x
Data Processing	Pandas	2.0.1
Numerical	NumPy	1.24.3
Database ORM	SQLAlchemy	1.4.48
MS SQL Driver	pyodbc	4.0.39
PostgreSQL Driver	psycopg2	Latest
Excel Reading	openpyxl	3.1.2
Config Management	python-dotenv	1.0.0
HTTP Requests	requests	2.27.1
Primary DB (OLTP)	MS SQL Server	SQLEXPRESS
Secondary DB	PostgreSQL	localhost
BI Tool	Power BI	—

---

4. Data Sources

The pipeline ingests data from 10+ external operational systems, all exported as flat files to designated network or local paths.

4.1 Ellis Property Management System (Ellis PMS)

Dataset	File Format	Source Path	Staging Table
All Units	Excel (.xlsm)	`Z:\BI_Reports\Home Office Groups - BI Reports`	`all_units_stage`
Lease Details	Excel (.xlsm)	Same multi-property folder	`lease_details_stage`
Lease Expiration/Renewal	Excel (.xlsm)	Same multi-property folder	`lease_expiration_stage`
Resident Demographics	Excel (.xlsm)	Same multi-property folder	`resident_demographics_stage`

Leasing Activity Detail	XML / CSV	`C:\...\Reports\Visits`	`lease_activity_report_new_stag`
AIRM Lease Report	CSV	`C:\...\Reports\AIRM_Lease_Report\`	`lease_report_stage`
AIRM Rent Summary	CSV	`C:\...\RawReports\AIRM_RentSummaryReport.CSV`	`unit_rent_summary_report_stage`
Unit Availability	Excel	`Z:\BI_Reports\...`	`unit_availbility_report_stage`
Unit Turnover	Excel	`Z:\BI_Reports\...`	`unit_turnover_stage`
Unit Amenities	CSV	`C:\...\BI_Amenities\`	`unit_amenties_report_stage`

4.2 Knock Lead Management System

Dataset	File Format	Source Path	Staging Table
Knock Activity (Daily)	CSV	`C:\...\BI_Knock_Activity_Daily\`	`knock_activity_feed_stage`
Knock Activity (Weekly)	CSV	`C:\...\BI_Knock_Activity_Weekly\`	`knock_activity_feed_stage_weekly`
Knock Conversion (Daily)	CSV	`C:\...\BI_Knock_Conversion_Daily\`	`knock_conversion_feed_stage`
Knock Engagement	CSV	`C:\...\RawReports\...`	`knock_engagement_feed_stage`

4.3 Finance & Accounting

Dataset	File Format	Source Path	Staging Table
Income Statements	CSV	`C:\...\PowerBIReportsFile\Reports\Income Statements-Current\`	`income_statement_stage`
Budget Statements	CSV	Same finance folder	`budget_statement_stage`
Delinquent/Prepaid	Excel	`Z:\BI_Reports\BI_Delinquent_Prepaid`	`delinquent_report_stage`
Banner Pending Invoices	Excel	`Z:\BI_Banner_Errors\...`	`bi_banner_pending_invoice_stage`
Banner Assigned Invoices	Excel	`Z:\BI_Banner_Errors\...`	`bi_banner_assigned_invoice_stage`
Banner DV Invoices	Excel	`Z:\BI_Banner_Errors\...`	`bi_banner_dv_invoice_stage`
Banner Error Reports	Excel	`Z:\BI_Banner_Errors\Multiple - Current Year Export.xlsx`	`bi_banner_error_report_stage`

4.4 Operations & Maintenance

Dataset	File Format	Source Path	Staging Tabl
Maintenance Requests	CSV / Excel	`Z:\BI_Reports\MaintenanceSummary`	`requests_reports_stage`
Service Requests	CSV	`Z:\BI_Reports\BI_Service_Request_List\`	`service_requests_report_st`
Lease Trade-out	CSV	`C:\...\Reports\LeaseTradeOut\LeaseTradeoutReport.csv`	`lease_trade_report_stage`
CAPEX Reports	TXT	`C:\...\BI_CAPEX\`	`unit_turn_capx_report_stag`
Rehab Reports	TXT	`C:\...\BI_REHAB\`	`unit_turn_rehab_report_sta`
Standard Turns	TXT	`C:\...\BI_TURNS\`	`unit_standard_turn_report_`
Weekly Reports	CSV	`C:\...\WeeklyRawReport\WeeklyReport`	`units_weekly_stats_stage`
Renewals (Friday)	Excel (.xlsm)	`C:\...\RenewalsFridayReports_CurrentRenewalStatus.xlsm`	`load_stage_friday_renewals`
Market Rate Reports	CSV	`C:\...\BI_Market_Rate\`	`market_rate_report_stage`
Renewal Offer Analysis	CSV	`C:\...\BI_RenewalOfferAnalysis_ROA\`	`unit_renewal_offer_analysi`

4.5 External Review & Engagement Platforms

Dataset	File Format	Source Path	Staging Table
Google Reviews	CSV	`C:\Desktop\Scripts\output.csv`	`google_review_stage`
Yelp Reviews	CSV	`C:\...\BI_Yelp_Review_Daily\`	`yelp_reviews_feed_stage`
SOCI Report Details	CSV	`C:\...\Reports\BI_Soci`	`soci_report_stage`
SOCI Review Feed	CSV	`C:\...\BI_Soci_Reviews\Review_Feed.csv`	`soci_review_feed_stage`
Ellis Loyalty (Weekly)	CSV	`C:\...\BI_Ellis_Loyalty_Current\`	`bi_ellis_loyality_feed_stage_weekly`
Ellis Loyalty (Monthly)	CSV	`C:\...\BI_Ellis_Loyalty_Current\`	`bi_ellis_loyality_feed_stage_monthly`

Ellis KPI (Weekly)	CSV	`C:\...\BI_Ellis_KPI_Current\`	`bi_ellis_kpi_feed_stage_weekly`
Ellis KPI (Monthly)	CSV	`C:\...\BI_Ellis_KPI_Current\`	`bi_ellis_kpi_feed_stage_monthly`

4.6 Rent Payment System

Dataset	File Format	Source Path	Staging Table
Rent Grata Weekly Report	CSV	`C:\...\BI_Rentgrata_Weekly_Report\`	`bi_weekly_rent_grata_report_stage`
Rent Grata Unpaid Rewards	CSV	`C:\...\BI_Rentgrats_Unpaid_Rewards\`	`bi_weekly_rent_grata_unpaid_rewards_stage`

4.7 Property Reference Data

Dataset	File Format	Source Path	Staging Table
Entities	CSV	`C:\...\Reports\Entities\Entities.csv`	`entities_stage`
Property Acquisitions	Excel (.xlsm)	`C:\...\Reports\Acquisitions\PropertyIAcquisition.xlsm`	`property_acquisition_stage`
Property Info	Excel (.xlsm)	`C:\...\RevenueManagementPropertyInfo.xlsm`	`property_info_stage`
Contact Level Details *(PostgreSQL only)*	CSV	`Z:\BI_Reports\...`	`contact_level_details`

---

5. ETL Pipeline Phases

The pipeline is divided into three sequential phases, orchestrated by Prefect flows:

```
workflow_tree_postgres.py::workflow_postgres()
│
├── [Phase 1]  load_staging()        — ConcurrentTaskRunner
│                                       30+ tasks run in PARALLEL
│
├── [Phase 2]  execute_dimensions_stored_procedures()  — SequentialTaskRunner
│                                       5 procedures run in SEQUENCE
│
```

```
└── [Phase 3]  execute_fact_stored_procedures()  — ConcurrentTaskRunner
                                                 40+ procedures run in PARALLEL
```

Phase 1: Extract & Stage (Concurrent)

Runner: `ConcurrentTaskRunner` **Goal:** Read all source files, apply raw transformations, and load into staging tables.

Each load task follows this pattern:

```
1. Connect to database (MSSQL or PostgreSQL)
2. TRUNCATE staging table(s)   ← fresh load every run
3. Read source file(s) from configured path
4. Instantiate transformation class (e.g., AllUnitsData)
5. Call structurize_data() to normalize the data
6. Batch INSERT into staging table (10,000 rows/batch)
7. Return task state
```

File type handling:

Format	Reader Method	Notes
Standard CSV	`read_csv_file()`	comma-delimited
Quoted CSV	`read_quote_csv_file()`	handles embedded commas
Tab-delimited	`read_csv_file_tab_delimited()`	CAPEX/Rehab/Turns files
Excel (.xlsx/.xlsm)	`read_excel_file()`	openpyxl engine
XML	`ready_activity_report_xml()`	Lease Activity visits
Multi-block CSV	`read_knock_conversion_csv()`	8-row block parser for Knock
Two-row CSV	`read_google_csv()`	custom Google Reviews format

Multi-file directory processing: Many load tasks scan an entire directory (e.g., `Z:\BI_Reports\Home Office Groups - BI Reports`) and process all matching files. File filtering excludes filenames containing `"zhistorical"` to skip archived data.

Phase 2: Dimension Population (Sequential)

Runner: `SequentialTaskRunner` **Goal:** Populate or update dimension/reference tables from the freshly loaded staging data.

These procedures must run sequentially because fact tables depend on dimension keys:

Order	Stored Procedure	Source Staging Table	Target Dimension Table
1	`SP_insert_update_Dim_info_property`	`property_info_stage`, `entities_stage`	`dim_info_property`
2	`SP_insert_dim_unit`	`all_units_stage`	`dim_unit`

3	`SP_insert_dim_resident_demographics`	`resident_demographics_stage`	`dim_resident_demographics`
4	`sp_populate_income_budget_categories`	`income_statement_stage`	`dim_income_budget_categories`
5	`SP_dim_requests`	`requests_reports_stage`	`dim_requests`

Also called separately (in some flows):

- `SP_insert_update_dim_info_property_history` — tracks property history changes
- `SP_insert_dim_requests_new` — newer requests dimension variant

Phase 3: Fact Table Population (Concurrent)

Runner: `ConcurrentTaskRunner` **Goal:** Execute all transformation stored procedures that read from staging/dimension tables and insert/update fact tables.

Each stored procedure:

1. Reads from the corresponding staging table(s)
2. Joins with dimension tables to resolve surrogate keys
3. Applies business logic (null handling, type casting, aggregation)
4. Performs UPSERT (INSERT or UPDATE) into the fact table
5. Creates snapshots for historical tracking where applicable

Complete fact procedure list:

Stored Procedure	Source Staging	Target F
`SP_insert_fact_availability_history`	`unit_availability_data_stage`	`fact_availabil`
`SP_insert_fact_turnover_history`	`unit_turnover_stage`	`fact_turnover_`
`SP_insert_update_fact_lease`	`lease_details_stage`	`fact_lease`
`SP_insert_update_fact_lease_all_units`	`all_units_stage`	`fact_lease_all_`
`SP_insert_update_fact_lease_all_units_history`	`all_units_stage`	`fact_lease_all_`
`SP_insert_update_fact_lease_expiration_renewal`	`lease_expiration_stage`	`fact_lease_exp`
`SP_populate_income_budget_fact_table` (type=1)	`budget_statement_stage`	`fact_income_bu`
`SP_populate_income_budget_fact_table` (type=2)	`income_statement_stage`	`fact_income_bu`
`SP_insert_fact_weekly_reports`	`units_weekly_stats_stage`	`fact_weekly_re`
`SP_insert_pending_renewals_history`	`load_stage_friday_renewals_data`	`fact_pending_r`
`SP_fact_delinquent_history`	`delinquent_report_stage`	`fact_delinquen`
`SP_insert_update_fact_lease_report`	`lease_report_stage`	`fact_lease_rep`
`SP_insert_update_fact_lease_tradeout_report`	`lease_trade_report_stage`	`fact_lease_tra`
`sp_insert_market_rent_report`	`market_rate_report_stage`	`fact_market_re`
`sp_insert_unit_amenities_hist`	`unit_amenties_report_stage`	`fact_unit_amen`

sp_insert_update_unit_turn_capx_report	unit_turn_capx_report_stage	fact_unit_turn_
sp_insert_update_unit_turn_rehab_report	unit_turn_rehab_report_stage	fact_unit_turn_
sp_insert_update_unit_turn_standard_report	unit_standard_turn_report_stage	fact_unit_turn_
sp_insert_unit_renewal_offer_analysis_report	unit_renewal_offer_analysis_report_stage	fact_renewal_o
sp_insert_update_lease_activity_detail	(legacy)	fact_lease_act
sp_insert_update_lease_activty_new	lease_activity_report_new_stage	fact_lease_act
sp_soci_report_details_snapshot	soci_report_stage	fact_soci_deta
sp_insert_update_soci_review_feed_details	soci_review_feed_stage	fact_soci_revi
sp_insert_knock_activity_daily_snapshot	knock_activity_feed_stage	fact_knock_act
sp_insert_knock_activity_weekly_snapshot	knock_activity_feed_stage_weekly	fact_knock_act
sp_insert_knock_conversion_daily_snapshot	knock_conversion_feed_stage	fact_knock_con
sp_insert_knock_engagement_weekly_snapshot	knock_engagement_feed_stage	fact_knock_eng
SP_olympus_lease_trend_analysis_procedure	Multiple fact tables	fact_lease_tre
sp_insert_update_ellis_weekly_monthly_loyality	bi_ellis_loyality_feed_stage_*	fact_ellis_loy
sp_insert_update_ellis_weekly_monthly_KPI	bi_ellis_kpi_feed_stage_*	fact_ellis_kpi
sp_insert_update_weekly_rent_grata_report	bi_weekly_rent_grata_report_stage	fact_weekly_re
sp_insert_update_weekly_rent_grata_unpaid_rewards	bi_weekly_rent_grata_unpaid_rewards_stage	fact_rent_grat
sp_insert_update_bi_banner_report	bi_banner_*_stage	fact_banner_re
sp_insert_dim_requests_new	service_requests_report_stage	dim_requests (n

---

6. Load Task Inventory

Complete list of all 33 Prefect load tasks:

#	Task Name	Source Type	Destination	Notes
1	load_acquisition_entities_propertyinfo	Excel	3 staging tables	Property reference data
2	load_all_units_lease_deatails_lease_expiration	Excel	3 staging tables	Core lease data
3	load_availability_turnover	Excel	2 staging tables	Unit operations
4	process_finance_statements_to_sql	CSV	2 staging tables	Income & budget

5	`load_weekly_files`	CSV	1 staging table	Weekly stats
6	`load_renewals_friday_reports`	Excel	1 staging table	Renewal pipeline
7	`load_request_files`	CSV/Excel	1 staging table	Maintenance
8	`load_delinquent_files`	Excel	1 staging table	Delinquency
9	`load_lease_report_files`	CSV	1 staging table	AIRM lease
10	`load_lease_tradeout_report_files`	CSV	1 staging table	Trade-out
11	`load_google_reviews`	CSV	1 staging table	Reviews
12	`load_market_rate_report`	CSV	1 staging table	Market rates
13	`load_unit_rent_summary_report`	CSV	1 staging table	Rent summary
14	`load_unit_amenities_report`	CSV	1 staging table	Amenities
15	`load_unit_capx_rehab_standard_turn`	TXT	3 staging tables	CAPEX/Rehab/Turns
16	`load_unit_renewal_offer_report`	CSV	1 staging table	ROA
17	`load_unit_availbility_report`	Excel	1 staging table	Availability
18	`load_service_requests_new`	CSV	1 staging table	Service tickets
19	`load_soci_report_details`	CSV	1 staging table	SOCI reports
20	`load_soci_review_details_new`	CSV	1 staging table	SOCI reviews
21	`load_knock_activity_report_new`	CSV	2 staging tables	Knock daily/weekly
22	`load_knock_conversion_report_new`	CSV	1 staging table	Knock conversion
23	`load_knock_engagement_report_new`	CSV	1 staging table	Knock engagement

24	`load_bi_ellis_loyality_report_new`	CSV	2 staging tables	Ellis loyalty W/M
25	`load_bi_ellis_kpi_report_new`	CSV	2 staging tables	Ellis KPI W/M
26	`load_rent_grata_unpaid_reward_report_new`	CSV	1 staging table	Rent Grata unpaid
27	`load_bi_weekly_rent_grata_report`	CSV	1 staging table	Rent Grata weekly
28	`load_yelp_review_details`	CSV	1 staging table	Yelp reviews
29	`load_bi_banners`	Excel	4 staging tables	Banner invoices
30	`load_lease_activity_new`	XML/CSV	1 staging table	Visit tracking
31	`load_resident_demographics`	Excel	1 staging table	Demographics
32	`load_contact_level_details_script` *(PG only)*	CSV	1 staging table	Contact details
33	`execute_stored_procedures`	—	Calls all SPs	Phase 2 & 3 dispatcher

---

7. Stored Procedures Reference

Location

- In the database server: `Olympus_Property_Staging` (MSSQL) / `staging` schema (PostgreSQL)
- Source file (one example checked in): `stored_procedures/SP_insert_update_fact_lease_expiration_renewal.sql`

Execution Pattern

All stored procedures are called via Python wrapper methods on the database connection class:

```
# MS SQL
ms_sql.execute_SP_insert_update_fact_lease_expiration_renewal()

# PostgreSQL
ps_sql.execute_SP_insert_update_fact_lease_expiration_renewal()
# → calls: CALL staging.SP_insert_update_fact_lease_expiration_renewal()
```

Example SP Logic (`SP_insert_update_fact_lease_expiration_renewal`)

```sql
-- 1. Create temporary table from staging
SELECT * INTO #temp_lease_expiration FROM lease_expiration_stage

-- 2. Join with dim_unit to resolve surrogate key
UPDATE #temp SET unit_id = d.unit_id
FROM #temp t JOIN dim_unit d ON t.unit_number = d.unit_number
              AND t.property_id = d.property_id

-- 3. Clean null placeholders
UPDATE #temp SET expiration_date = NULL WHERE expiration_date = 'None'
UPDATE #temp SET renewal_date   = NULL WHERE renewal_date   = 'None'

-- 4. Type cast
ALTER TABLE #temp ALTER COLUMN rent MONEY
ALTER TABLE #temp ALTER COLUMN expiration_date DATE

-- 5. Insert/Update fact table
MERGE fact_lease_expiration_renewal AS target
USING #temp AS source
ON target.lease_id = source.lease_id
WHEN MATCHED THEN UPDATE SET ...
WHEN NOT MATCHED THEN INSERT ...

-- 6. Cleanup
DROP TABLE #temp_lease_expiration
```

---

8. Data Transformation Classes

All transformation logic lives in `transform_data/data.py`. A base `Data` class provides shared file-reading utilities; each subclass implements `structurize_data()` for its specific source format.

Base Class Methods

Method	Purpose
`read_csv_file()`	Standard comma-delimited CSV
`read_quote_csv_file()`	CSV with quoted fields
`read_csv_file_tab_delimited()`	Tab-separated files
`read_excel_file()`	Excel files via openpyxl
`read_google_csv()`	Two-header-row Google export format
`read_knock_conversion_csv()`	8-row block structure parser
`ready_activity_report_xml()`	XML lease activity files
`replace_nan_values()`	Convert pandas NaN → Python None (→ SQL NULL)

Transformation Subclasses

Class	Source System	Key Operations
`EntityData`	Ellis / Admin	Reads entity hierarchy, normalizes names
`AcquisitionData`	Acquisitions	Reads acquisition timeline and financials
`PropertyInfoData`	Revenue Mgmt	Property attributes and metadata
`AllUnitsData`	Ellis PMS	Multi-file concat, column standardization
`LeaseDetailData`	Ellis PMS	Lease terms, resident linkage
`LeaseExpirationData`	Ellis PMS	Expiration dates, renewal flags
`ResidentDemographicsData`	Ellis PMS	Demographics, screening info
`AvailabiltyData`	Ellis PMS	Unit availability status
`TurnoverData`	Ellis PMS	Move-in/move-out tracking
`IncomeBudgetData`	Finance	Multi-file income/budget parsing
`WeeklyReports`	Ellis PMS	Occupancy & leasing stats
`RenewalsFridayReports`	Ellis PMS	Renewal pipeline Excel data
`MaintainenceReports`	Maintenance	Work order parsing
`DelinquentReportsNew`	Finance/Ellis	Balance aging data
`LeasereportData`	AIRM	Lease analytics
`LeasetradereportData`	AIRM	Lease trade-out pricing
`GoogleReviews`	Google	Rating/review parsing
`MarketRateReportData`	Market Research	Competitor rent data
`UnitAmenitiesReport`	Ellis PMS	Unit feature catalog
`UnitRentSummaryReport`	AIRM	Per-unit rent breakdown
`UnitCapXReport`	CAPEX	Capital project tracking
`UnitRehabReport`	CAPEX	Rehab project tracking
`UnitStandardTurnReport`	Operations	Standard turn procedures
`UnitRenewalOfferAnalysis`	Revenue Mgmt	Renewal pricing analysis
`LeaseActivityData` (legacy)	Ellis PMS	Activity history
`LeaseActivityVisits`	Ellis PMS	Visit/traffic tracking (XML)
`UnitAvailbilityData`	Ellis PMS	Detailed availability
`ServiceRequestData`	Maintenance	Service ticket parsing
`ResidentDetails`	Ellis PMS	Contact demographics

`SOCIReportDetails`	SOCI	Reputation report data
`SOCIReviewFeed`	SOCI	Review feed with ratings
`KnockActivityDaily`	Knock	Daily lead activity
`KnockCoversionReport`	Knock	Conversion funnel metrics
`YELPReviewFeed`	Yelp	Yelp ratings and reviews
`BIEllisLoyaltyDataWeekly`	Ellis	Weekly loyalty scores
`BIEllisLoyaltyDataMonthly`	Ellis	Monthly loyalty scores
`BIEllisKPIDataWeekly`	Ellis	Weekly KPI metrics
`BIEllisKPIDataMonthly`	Ellis	Monthly KPI metrics
`EllisSurveyReport`	Ellis	Survey responses
`KnockEngagementReportDaily`	Knock	Daily engagement stats
`RentGrataWeeklyReport`	Rent Grata	Weekly payment data
`RentGrataUnpaidRewards`	Rent Grata	Unpaid reward tracking
`BIBannersErrors`	Banner	Invoice error logs
`BIBannersPendingInvoice`	Banner	Pending invoice details
`BIBannersAssignedInvoice`	Banner	Assigned invoice details
`BIBannersDVInvoice`	Banner	DV invoice details

---

9. Database Connection Layer

MSSQL Class (`database/ms_sql.py`)

```
Connection: SQLAlchemy + pyodbc
Driver:     ODBC Driver 17 for SQL Server
Auth:       Windows Trusted Connection
Server:     CCPOLYSQL01\SQLEXPRESS
Database:   Olympus_Property_Staging
```

Key Methods:

Method	Description
`connect()`	Establishes SQLAlchemy engine and connection
`execute_query(sql)`	Runs any DDL/DML statement
`truncate_tables([table_names])`	Truncates one or more staging tables
`insert_dataframe(table, df)`	Batch-inserts DataFrame in 10,000-row chunks

`execute_SP_<name>()`	40+ wrapper methods, one per stored procedure

PostgreSQL Class (`database/ps_sql.py`)

```
Connection: SQLAlchemy + psycopg2
Server:     localhost
Database:   Olympus_Property
Schema:     staging
User:       postgres
Pool:       pre_ping=True (connection health check)
```

Key Methods:

Method	Description
`connect()`	Establishes connection with schema path
`insert_dataframe(table, df)`	Standard multi-row insert
`insert_dataframe_copy_command(table, df)`	COPY FROM (faster bulk load)
`insert_dataframe_copy_command_contact_details(table, df)`	Specialized COPY for contact CSV format
`refresh_pbi_web_ai_views()`	Refreshes PostgreSQL materialized views
`execute_SP_<name>()`	40+ wrapper methods calling `CALL staging.<proc>()`

Insert Performance

Method	Use Case	Speed
Standard `insert_dataframe`	General purpose	Moderate
`insert_dataframe_copy_command`	Large CSV loads	Faster (uses PostgreSQL COPY)
Batch size = 10,000 rows	All insert methods	Prevents memory overflow

---

10. Scheduling & Deployment

Prefect Deployment Files

MS SQL Pipeline (`workflow-deployment.yaml`)

```yaml
name: olympus_workflow_tree
entrypoint: workflow_tree.py:workflow
work_pool_name: olympus_workpool
work_queue_name: olympus_workqueue
schedule:
  interval: 86400        # Every 24 hours
  anchor_date: 2023-07-12T13:15:00+00:00
```

```yaml
  timezone: Asia/Karachi
infrastructure: process
is_schedule_active: true
```

PostgreSQL Pipeline (`workflow_postgres-deployment.yaml`)

```yaml
name: olympus_workflow_postgres
entrypoint: workflow_tree_postgres.py:workflow_postgres
work_pool_name: olympus_workpool
work_queue_name: olympus_workqueue
schedule:
  interval: 86400        # Every 24 hours
  anchor_date: 2026-01-14T14:00:00+00:00
  timezone: Asia/Karachi
infrastructure: process
is_schedule_active: true
```

Deployment Summary

Property	MS SQL	PostgreSQL
Flow Name	`olympus_workflow_tree`	`olympus_workflow_postgres`
Work Pool	`olympus_workpool`	`olympus_workpool`
Work Queue	`olympus_workqueue`	`olympus_workqueue`
Schedule	Daily @ ~13:15 UTC	Daily @ ~14:00 UTC
Timezone	Asia/Karachi (UTC+5)	Asia/Karachi (UTC+5)
Infrastructure	Process (local)	Process (local)
Entry Point	`workflow_tree.py:workflow`	`workflow_tree_postgres.py:workflow_postgres`

Manual Execution

Manual runs are supported via:

- `manual_workflow.py` — MS SQL one-off run
- `manual_workflow_postgres.py` — PostgreSQL one-off run

---

11. Directory Structure

```
Olympus-property-analytics-master/
│
├── workflow_tree.py                    ← Main MS SQL Prefect flow
├── workflow_tree_postgres.py           ← Main PostgreSQL Prefect flow
├── workflow_tree_test_version.py       ← Test/dev flow variant
├── workflow_tree_backup.py             ← Backup copy of main flow
├── manual_workflow.py                  ← MS SQL manual execution
├── manual_workflow_postgres.py         ← PostgreSQL manual execution
```

```
│
├── workflow-deployment.yaml            ← Prefect deployment (MS SQL)
├── workflow_postgres-deployment.yaml   ← Prefect deployment (PostgreSQL)
│
├── database/
│   ├── __init__.py
│   ├── ms_sql.py                       ← MS SQL connection & SP wrappers
│   ├── ps_sql.py                       ← PostgreSQL connection & SP wrappers
│   └── snowflake.py                    ← Snowflake (present, not active)
│
├── prefect_ms_sql_load/                ← MS SQL load tasks (30+ files)
│   ├── __init__.py                     ← Exports all task functions
│   ├── execute_stored_procedures.py    ← SP execution tasks
│   ├── process_finance_statements_to_sql.py
│   └── load_*.py                       ← One file per data source
│
├── prefect_ps_sql_load/                ← PostgreSQL load tasks (30+ files)
│   ├── __init__.py                     ← Exports all task functions
│   ├── execute_stored_procedures.py    ← SP execution tasks (Postgres)
│   ├── variable.py                     ← Contact level schema mappings
│   ├── load_contact_level_details.py   ← PostgreSQL-only task
│   └── load_*.py                       ← One file per data source
│
├── transform_data/
│   ├── __init__.py
│   └── data.py                         ← 45+ transformation classes
│
├── stored_procedures/
│   └── SP_insert_update_fact_lease_expiration_renewal.sql
│
├── push_lease_activity_report_to_csv.py
├── push_all_lease_activity_report_to_csv.py
├── push_all_private_lease_activity_report_to_csv.py
├── push_all_fact_lease_all_units_report_to_csv.py
├── push_Vena_lease_activity_report_to_csv.py
│
├── get_stat_report_monthly.py          ← Monthly statistics reporting
├── requirements.txt                    ← Python dependencies
├── .env                                ← Environment variables (not committed)
└── error_log.txt                       ← Pipeline error log
```

---

12. Environment Configuration

All configuration is managed via `.env` file and loaded with `python-dotenv`.

Database Connection Variables

```
# MS SQL Server
SERVER_NAME     = CCPOLYSQL01\SQLEXPRESS
DATABASE_NAME   = Olympus_Property_Staging
```

```
# PostgreSQL
PS_SERVER_NAME  = localhost
PS_DATABASE_NAME= Olympus_Property
PS_USER_NAME    = postgres
PS_PASSWORD     = Olympus4
PS_SCHEMA       = staging
```

Staging Table Name Variables (sample)

```
ALL_UNITS_TABLE_NAME                    = all_units_stage
LEASE_DETAILS_TABLE_NAME                = lease_details_stage
LEASE_EXPIRATION_TABLE_NAME             = lease_expiration_stage
RESIDENT_DEMOGRAPHICS_TABLE_NAME        = resident_demographics_stage
INCOME_STATEMENT_TABLE_NAME             = income_statement_stage
BUDGET_STATEMENT_TABLE_NAME             = budget_statement_stage
WEEKLY_UNITS_TABLE_NAME                 = units_weekly_stats_stage
DELINQUENT_REPORT_TABLE_NAME            = delinquent_report_stage
GOOGLE_REVIEW_TABLE_NAME                = google_review_stage
KNOCK_ACTIVITY_TABLE_NAME               = knock_activity_feed_stage
KNOCK_CONVERSION_TABLE_NAME             = knock_conversion_feed_stage
SOCI_REPORT_TABLE_NAME                  = soci_report_stage
YELP_REVIEWS_TABLE_NAME                 = yelp_reviews_feed_stage
# ... 40+ additional table name variables
```

File Path Variables (sample)

```
MULTIPLE_PROPERTY_DATA      = Z:\BI_Reports\Home Office Groups - BI Reports
WEEKLY_FILE_PATH            = C:\Users\revenuemanagement\OneDrive - Olympus Property\...
LEASE_TRADEOUT_FILE_PATH    = C:\...\Reports\LeaseTradeOut\LeaseTradeoutReport.csv
GOOGLE_REVIEWS_FILE_PATH    = C:\Desktop\Scripts\output.csv
KNOCK_ACTIVITY_FILE_PATH    = C:\...\BI_Knock_Activity_Daily\
MARKET_RATE_FILE_PATH       = C:\...\BI_Market_Rate\
# ... 30+ additional file path variables
```

---

13. Error Handling & Logging

Log Files

Log File	Location	Content
`error_log.txt`	Project root	General pipeline errors
`load_<task_name>.log`	Project root	Per-task load errors
Prefect UI	Web dashboard	Full task run history and stack traces

Error Handling Strategy

- Each load task is wrapped as a Prefect `@task` — individual task failure does not stop the entire flow
- File reading errors are caught and logged; the task returns a failed state
- Batch inserts: if one batch fails, the error is logged and execution continues with the next batch
- Staging tables are TRUNCATED at the start of each run, ensuring stale data is not carried over
- Null/empty string handling: `'None'` string literals and `''` are converted to SQL `NULL` before insertion

Data Quality Checks

- `replace_nan_values()` — converts pandas `NaN` / `NaT` to Python `None`
- Type validation via stored procedure `ALTER TABLE ... ALTER COLUMN` (fail-fast on bad data types)
- File existence and path validation at the start of each load task

---

14. Data Flow Diagram

```
╔═══════════════════════════════════════════════════════════════════╗
║                    DAILY ETL RUN (Asia/Karachi timezone)          ║
╚═══════════════════════════════════════════════════════════════════╝


┌──────────────────────────────────────────────────────────────┐
│  EXTERNAL SYSTEMS  (files written to network/local paths)      │
│                                                                │
│  Ellis PMS │ Knock │ Banner │ Finance │ SOCI │ Yelp │ Google  │
│  Market │ Maintenance │ Rent Grata │ Ellis KPI │ Ellis Loyalty │
└──────────────────────────────┬──────────────────────────────┘
                               │ CSV / Excel / XML / TXT files
                               ▼
┌──────────────────────────────────────────────────────────────┐
│  PHASE 1: EXTRACT & LOAD STAGING (ConcurrentTaskRunner)        │
│  Duration: ~5-15 min                                           │
│                                                                │
│  transform_data/data.py                                        │
│  ├── Read files (Pandas)                                       │
│  ├── Normalize columns                                         │
│  ├── Replace nulls                                             │
│  └── Batch insert → STAGING TABLES (10,000 rows/batch)        │
│                                                                │
│  [30+ tasks run in PARALLEL]                                   │
└──────────────────────────────┬──────────────────────────────┘
                               │
                               ▼
┌──────────────────────────────────────────────────────────────┐
│  STAGING TABLES (50+ tables, all prefixed _stage)              │
│                                                                │
│  all_units_stage │ lease_details_stage │ lease_expiration_stage│
│  income_statement_stage │ knock_activity_feed_stage │ ...      │
└──────────────────────────────┬──────────────────────────────┘
                               │
                               ▼
┌──────────────────────────────────────────────────────────────┐
│  PHASE 2: DIMENSION POPULATION (SequentialTaskRunner)          │
│  Duration: ~2-5 min                                            │
```

```
│                                                                │
│  Stored Procedures:                                            │
│  1. SP_insert_update_Dim_info_property                         │
│  2. SP_insert_dim_unit                                         │
│  3. SP_insert_dim_resident_demographics                        │
│  4. sp_populate_income_budget_categories                       │
│  5. SP_dim_requests                                            │
└────────────┬──────────────────────────────────────┬────────────┘
             │                                      │
             ▼                                      ▼

┌──────────────────────────┐              ┌──────────────────────────┐
│   DIMENSION TABLES       │              │   DIMENSION TABLES       │
│   dim_info_property      │              │   (same set)             │
│   dim_unit               │              │                          │
│   dim_resident_demo...   │              │                          │
│   dim_income_budget_cat  │              │                          │
│   dim_requests           │              │                          │
└────────────┬─────────────┘              └────────────┬─────────────┘
             └──────────────────────────┬──────────────┘
                                        ▼

┌────────────────────────────────────────────────────────────────┐
│  PHASE 3: FACT TABLE POPULATION (ConcurrentTaskRunner)         │
│  Duration: ~15-30 min                                          │
│                                                                │
│  40+ Stored Procedures running in PARALLEL                     │
│  Each: READ staging → JOIN dims → UPSERT fact                 │
└────────────────────────────┬───────────────────────────────────┘
                             │
                             ▼

┌────────────────────────────────────────────────────────────────┐
│  FACT TABLES (40+ tables)                                      │
│                                                                │
│  fact_lease │ fact_lease_expiration_renewal │ fact_lease_report│
│  fact_availability_history │ fact_turnover_history            │
│  fact_income_budget │ fact_weekly_reports                      │
│  fact_delinquent_history │ fact_market_rent_report             │
│  fact_knock_activity_daily │ fact_soci_details_snapshot        │
│  fact_ellis_kpi │ fact_ellis_loyalty │ ... (30+ more)         │
└────────────────────────────┬───────────────────────────────────┘
                             │
                             ▼

┌────────────────────────────────────────────────────────────────┐
│  CONSUMPTION LAYER                                             │
│                                                                │
│  Power BI Dashboards       push_*.py → CSV Exports             │
│  Revenue Analytics         get_stat_report_monthly.py          │
│  Operational Reports       Ad-hoc SQL Queries                  │
└────────────────────────────────────────────────────────────────┘


Total Pipeline Duration: ~30-60 minutes per daily run
```

Document generated from codebase analysis of Olympus-property-analytics-master. For questions about this pipeline contact the Analytics / Revenue Management team.